

CS 6888: Program Analysis and its Applications

Assignment 3: Fault Isolation

Due: March 4th at 5 PM
Worth 10 points

1 Learning Objective

To consolidate your understanding of, and obtain hands-on experience with, **fault-isolation techniques**, including:

- how they work
- their strengths and weaknesses

2 Individual Assignment

This is an individual assignment. You are not to share your setup, strategies, results, analysis, code, or any other aspect of the assignment with your classmates.

3 Preparation

3.1 Background

This assignment will use `gcov`, a coverage measurement tool, to test modified versions of `tcas`. (`tcas` is part of an aircraft collision avoidance system originally crafted by Siemens researchers to experiment with testing techniques. Although just ~150 lines long, it comes with a large test suite.)

You have been provided with 5 versions of `tcas`. `tcas0.c` will pass all tests, while `tcas1.c` - `tcas4.c` each have a single fault that has been added with an accompanying comment (e.g., `// Mutant: added '- NOZCROSS'`). After you've compiled the respective versions of `tcas`, the test suite can be executed by running the `run_tests.sh` bash script with the variant number as the first argument (so `./run_tests.sh 2` to test `tcas2` compiled from `tcas2.c`). Passing tests will print a P, while failing tests will print an F.

3.2 Practice

Before you start the assignment, you should familiarize yourself with `gcov`, especially the output format of the `.gcov` files.

- <https://en.wikipedia.org/wiki/Gcov>
- <https://gcc.gnu.org/onlinedocs/gcc/Gcov-Intro.html#Gcov-Intro>
- <https://gcc.gnu.org/onlinedocs/gcc/Invoking-Gcov.html#Invoking-Gcov>

Before moving on, make sure you can compile `tcas`, instrument it to collect coverage information with `gcov`, and execute the provided tests on `tcas` while collecting coverage before proceeding.

Note: `llvm` isn't allowed for this assignment. (The method can be performed using `llvm` instead of `gcc` without issue generally, but the outputs will be slightly different so for the assignment use `gcc`.)

4 Tasks

4.1 Implement DStar.

A fault localization technique takes a program and a test suite (with at least one failing test) as input and produces a sorted list of suspicious lines of code. In class, we saw that there are many ways to compute suspiciousness¹. In this assignment, we're going to use the DStar technique, which defines a statement's suspiciousness as:

$$S(s) = \frac{failed(s)^2}{passed(s) + total_failed - failed(s)} \quad (1)$$

Where `failed(s)` is the number of failing tests executing statement `s`, `passed(s)` is the number of tests that executed `s` and passed, and `total_failed` is the total number of tests that failed.

To facilitate the computation of DStar, it is convenient to have the coverage information organized. Building on `run_tests.sh`, develop a script so that the coverage information for each executed test is stored in a separate file. The `run_tests.sh` script provided does not run `gcov`. You should extend the script to gather test coverage and store the coverage information for passing tests and failing tests in separate directories.

Tip: make sure to start with a “clean” set of coverage files when you run each test as the coverage counts are accumulated by default. You are welcome to move files around so that each version of `tcas` lives in its own folder, but be sure to use the same names for each version of `tcas` as outlined above.

In this assignment, you will implement a tool called `fl_dstar` (for fault localization with DStar). Your tool will take two files as command-line arguments: `passing_dir` and `failing_dir`. These arguments are the absolute paths to directories containing the `.gcov` files corresponding to the passing and failing tests respectively. You may use any language to create this tool, but your `run_all.sh` script must install any necessary programs from the apt repo if they aren't part of the clean image.

`fl_dstar` must analyze the coverage data in the proper `.gcov` files and consider whether the tests are passing or failing to then compute the DStar suspiciousness rating for each applicable program line. It should then print out a sorted list of program statements in descending order of suspiciousness values. The output should be in the exact following format: line number for statement `s`, quote of statement `s`, `#failedTests(s)`, `#passedTests(s)`, `total_failed`, `S(s)` Below is a truncated table of the results you should obtain for the `tcas2.c` version. Use these results as a sanity check to see if your `fl_dstar` implementation is working correctly. Note that if multiple lines have the same score, e.g. #59 and #61 below, then you have to sort them by the line number in ascending order. The line numbering should start at 1 and the main method will be on line 142 in all of the files. Round your scores to 2 decimals.

Line #	Statement	#failedTests(s)	#passedTests(s)	total_failed	Suspiciousness
136	<code>alt_sep = UPWARD_RA; // Mutant</code>	616	0	617	379456.00
133	<code>else if (need_downward_RA)</code>	616	123	617	3060.13
59	<code>int Inhibit_Biased_Climb ()</code>	617	267	617	1425.80
61	<code>return (Climb_Inhibit ? Up_Se ...</code>	617	267	617	1425.80

4.2 Run DStar

Run your `fl_dstar` tool on coverage generated from running each of `tcas1-tcas4` against the test suite.

4.3 Write a report.

The report must be under 3 pages in `ieee-conference` format² and include the following contents:

- For each of the 4 versions (`tcas1-tcas4`), include a table of the top 10 suspicious statements³ and answer the following questions:
 - Did `fl_dstar` identify the faulty statement as the most suspicious? (If not, explain why in terms of the coverage pattern and test results.)
 - Assuming you did not know the fault location and you were the developer in charge of fixing the unknown fault: how would the results of `dstar` affect your search for the fault?

¹If you want to learn more about the techniques available and how they fair on a variety of settings take a look at Spencer Pearson et al. “Evaluating and Improving Fault Localization”. In: *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. 2017, pp. 609–620. DOI: 10.1109/ICSE.2017.62

²Templates are available at <https://www.ieee.org/conferences/publishing/templates.html> or <https://www.overleaf.com/latex/templates/ieee-conference-template/grfzhncsfqn>.

³You can simply include images of the tool output as long as it is clearly legible if you don't want to make a latex table.

2. Conjecture and explain when this tool is likely to perform well and when it is likely to perform poorly.

5 Submission

5.1 Report

Submit your report through Canvas.

5.2 Code

Your repo will be cloned at the due date. Be sure that you have committed and pushed your code at that time.